

Solving the Imbalanced Problem by Metric Learning and Oversampling

Kaixiang Yang^{ID}, *Member, IEEE*, Zhiwen Yu^{ID}, *Senior Member, IEEE*, Wuxing Chen, Zefeng Liang, and C. L. Philip Chen^{ID}, *Fellow, IEEE*

Abstract—Imbalanced data poses a substantial challenge to conventional classification methods, which often disproportionately favor samples from the majority class. To mitigate this issue, various oversampling techniques have been deployed, but opportunities for optimizing data distributions remain underexplored. By exploiting the ability of metric learning to refine the sample distribution, we propose a novel approach, Imbalance Large Margin Nearest Neighbor (ILMNN). Initially, ILMNN is applied to establish a latent feature space, pulling intra-class samples closer and distancing inter-class samples, thereby amplifying the efficacy of oversampling techniques. Subsequently, we allocate varying weights to samples contingent upon their local distribution and relative class frequency, thereby equalizing contributions from minority and majority class samples. Lastly, we employ Kullback-Leibler (KL) divergence as a safeguard to maintain distributional similarity to the original dataset, mitigating severe intra-class imbalances. Comparative experiments on various class-imbalanced datasets verify that our ILMNN approach yields superior results.

Index Terms—Imbalance learning, metric learning, oversampling, sample reweighing.

I. INTRODUCTION

CLASS imbalance poses a significant challenge in classification tasks. Many real-world applications exhibit imbalanced class distributions, including power system evaluation [1], disease diagnosis [2], [3], software defect prediction [4], [5], [6], [7] and image recognition [8], [9]. In imbalanced datasets, the majority class has a greater impact due to its significantly

larger number of samples compared to the minority class. Consequently, the classifier tends to favor the majority class and suppress the minority class, resulting in a decision boundary biased towards the majority class. When a classifier is trained on an imbalanced datasets, the overall accuracy is frequently good, but the performance on the minority class may be noticeably poorer. For instance, when applying a traditional classifier to a dataset with an imbalance ratio of 100:1, the majority class is often classified for most samples, resulting in satisfactory accuracy. However, the majority of samples from the minority class are likely to be misclassified as the majority class [10]. In other words, the accuracy of the minority class will become very low. Therefore, learning from imbalanced datasets has become an important research topic in the field of machine learning.

Numerous methods [10] have been proposed to mitigate the adverse effects of imbalanced datasets. For example, cost-sensitive methods utilize distinct cost matrices for misclassified samples [11]. While the cost-sensitive method can enhance classifier performance, outliers and noise significantly impact cost-sensitive methods. Resampling is another popular method used to balance the distribution of different classes by generating or removing samples [12]. Under-sampling methods eliminate samples from the majority class, but this removal results in information loss, especially when the minority class samples are significantly fewer than the majority class samples. Oversampling creates additional minority samples to balance the sample distribution across each class. However, oversampling [13] is not without its challenges as it can introduce synthetic samples in overlapping areas, some of which may not contribute significantly to subsequent classification tasks. To solve this problem, some methods clean up the samples near the decision boundary after oversampling or perform processing and enhancement at the feature level [14], [15], [16]. Alternatively, other algorithms incorporate clustering techniques to enhance the performance of oversampling [17]. Furthermore, ensemble learning methods like boosting and bagging have been introduced [13], [18], [19]. Oversampling methods commonly employ the k -NN classifier to aid in sample selection. The k -NN classifier relies on the local distribution of samples. Hence, an improved local distribution is advantageous for oversampling methods.

Recently, there has been a growing focus among researchers on metric learning [3], [20], which aims to learn a feature space where the distances between samples are more meaningful. This type of approach has found applications in various real-world scenarios, such as person re-identification [21], image

Manuscript received 18 September 2023; revised 22 May 2024; accepted 22 June 2024. Date of publication 27 June 2024; date of current version 13 November 2024. This work was supported in part by the National Key R&D Program of China under Grant 2023YFA1011601, in part by the National Natural Science Foundation of China under Grant 62476101, Grant 62106224, and Grant U21A20478, in part by the Major Key Project of PCL, China under Grant PCL2023AS7-1 and Grant PCL2023A09, in part by Guangzhou Science and Technology Plan Project under Grant 2024A04J3749 and in part by the Fundamental Research Funds for the Central Universities under Grant 2024ZYGXZR062. Recommended for acceptance by X. Zhu. (Corresponding author: Zhiwen Yu.)

Kaixiang Yang and Zefeng Liang are with the School of Computer Science and Engineering, South China University of Technology, Guangzhou 511442, China (e-mail: yangkx@scut.edu.cn; zefliang@qq.com).

Zhiwen Yu is with the School of Computer Science and Engineering, South China University of Technology, Guangzhou 511442, China, and also with the Pengcheng Lab, Shenzhen 518066, China (e-mail: zhwyu@scut.edu.cn).

Wuxing Chen is with the Pengcheng Lab and the School of Future Technology, South China University of Technology, Guangzhou 511442, China (e-mail: fichenwuxing@mail.scut.edu.cn).

C. L. Philip Chen is with the School of Computer Science and Engineering, South China University of Technology, Guangzhou 511442, China, and also with the Pazhou Lab, Guangzhou 510335, China (e-mail: philip.chen@ieee.org).

Digital Object Identifier 10.1109/TKDE.2024.3419834

retrieval [22], and recommendation systems [23]. Metric learning algorithms specifically aim to learn a feature space where similar samples are closer together, while dissimilar samples are more separated. By mapping the original data to the learned feature space, the distance relationships between samples can be optimized, leading to improved classifier performance. One traditional metric learning method is Large Margin Nearest Neighbor (LMNN) [24], which is based on relative distance constraints. LMNN constructs a feature space based on the local distribution of samples, thereby enhancing the performance of the k -NN classifier. Some researchers have explored the application of metric learning to imbalanced datasets. However, even in the feature space learned by metric learning, the number of samples remains imbalanced, which adversely affects the classification task. Traditional metric learning methods usually do not consider their integration with oversampling strategies. The feature space created by these methods may not be fully conducive to oversampling. To harness the benefits of both metric learning and oversampling, oversampling is applied to the feature space learned by metric learning. Given that LMNN can optimize the local distribution of samples, it has the potential to improve the performance of oversampling. However, traditional metric learning methods have not considered their combination with oversampling. The feature space learned by traditional metric learning methods may hinder the effectiveness of oversampling methods. Furthermore, metric learning itself is prone to imbalance problems. Therefore, it becomes necessary to appropriately modify the original LMNN for subsequent oversampling methods.

In this study, we present an approach called Imbalance Large Margin Nearest Neighbor (ILMNN), which leverages metric learning to address imbalanced datasets. ILMNN initially learns a feature space that emphasizes proximity between samples of the same class while increasing separation between samples from different classes. This feature space proves advantageous in enhancing the performance of oversampling methods, particularly when dealing with imbalanced datasets. Additionally, we introduce a sample re-weighting strategy based on the loss function of the LMNN algorithm. This strategy assigns higher weights to minority class samples and samples that are prone to misclassification. Furthermore, we incorporate KL divergence to preserve the original local structure within the feature space and mitigate intra-class imbalance issues. To optimize the ILMNN algorithm, a dedicated optimization procedure is designed. Consequently, we obtain a feature space that is well-suited for oversampling tasks.

The main contributions of this paper are summarized by:

- 1) We propose the Imbalance Large Margin Nearest Neighbor (ILMNN) algorithm, which leverages metric learning, to discover a feature space suitable for oversampling.
- 2) We introduce a re-weighting strategy that assigns appropriate weights to key samples, including minority-class samples and samples that are easily misclassified, thereby further refining the learned feature space.
- 3) We incorporate KL divergence to impose constraints on the distance between the distribution in the feature space and the distribution of data in the original space, thereby providing benefits to oversampling methods.

The remainder of this work is organized as follows. Section II reviews related literature on resampling and metric learning. Section III introduces ILMNN and its corresponding optimization algorithm, along with an analysis of the time complexity. The experiment result and analysis are presented in Section IV. Finally, Section V concludes the paper and discusses potential avenues for future research.

II. RELATED WORK

In this section, we review related literature on resampling methods, metric learning and other methods of imbalanced learning.

A. Resampling Method

Resampling techniques have been widely employed to address the imbalanced distribution of datasets, typically categorized as under-sampling and over-sampling approaches. Random under-sampling [25] is an example of an under-sampling method where majority class samples are randomly removed, and the remaining majority class samples are combined with the minority class to form the training set. Similarly, Tomek Link [26] eliminates samples and their nearest neighbors if they belong to different classes. While under-sampling can rebalance the sample distribution and reduce the complexity of the learning process, it may result in information loss from the majority class. When there is a significant disparity in the number of samples across different classes, removing various majority samples can lead to substantial information loss.

To mitigate this issue, ensemble learning techniques are adopted to enhance under-sampling methods. For example, EasyEnsemble [27] samples multiple majority subsets and combines them with minority class samples to create training sets, upon which multiple classifiers are trained. BalanceCascade, on the other hand, progressively removes majority samples that are correctly classified by the current trained classifier and combines classifiers through an AdaBoost ensemble approach [27].

Over-sampling methods are employed to address imbalanced distributions by generating minority class samples. Random oversampling [25] replicates randomly selected minority samples. Although this method can mitigate the imbalanced problem, it carries the risk of overfitting. Subsequently, Synthetic Minority Over-sampling Technique (SMOTE) [28] is proposed, which generates samples between each minority class sample and its randomly chosen neighbors using linear interpolation, as defined below:

$$x_{gen} = x_{original} + (x_{neighbour} - x_{original}) * r, \quad (1)$$

where x_{gen} denotes the synthetic sample generated, $x_{original}$ represents the original sample selected, typically from one of the minority classes, $x_{neighbour}$ signifies a neighbouring sample of 'b' also selected from a minority class, and r is a random number, generally ranging between 0 and 1. SMOTE aims to randomly insert a new sample point between the original sample and its neighbours, thus increasing the number of samples in a few classes and improving the class balance of the training data.

ADASYN [29] assigns different sampling weights to minority class samples, enabling the synthesis of more challenging samples. MWMOTE [30] assigns larger weights to minority class samples that are closer to majority class samples. MDO [31] initially generates samples in the principal component space and then transforms them back to the Euclidean space. Entropy-based sampling [32] generates samples to balance the entropy of each class. Some studies combine oversampling with under-sampling techniques. CHC [33] generates minority samples while simultaneously under-sampling the majority subset divided by a clustering method. k -INOS [12] performs oversampling to remove potential noisy samples in the minority subsets, and then combines synthetic minority class samples to create a new training set. However, when datasets exhibit highly overlapping structures between different classes, oversampling may generate samples in the overlapping area, leading to negative effects on classifier performance.

B. Metric Learning

The performance of machine learning methods is often contingent upon the selection of suitable distance metrics. One commonly used metric is the Euclidean distance, which treats all features as equal and independent. However, it is normal that certain features may be important than others. For example, when determining the face image, features like hair colours and face shapes should be emphasised. Consequently, the Euclidean distance may inadequately assess the similarity between samples, thereby diminishing classifier performance. Metric learning addresses this issue by acquiring a task-specific distance measure. With a learned distance metric, similarity can be more effectively quantified by measuring the distance between samples, leading to enhanced classifier performance. A prominent area of research within metric learning focuses on the development of a more appropriate Mahalanobis distance [34], which is represented by:

$$\mathcal{L}(\mathbf{M}) = \text{Loss}(\mathbf{M}) + \lambda \text{Reg}(\mathbf{M}), \quad (2)$$

The first term is the loss function concerning measurement matrix $\mathbf{M}$, and the second part is the regularisation concerning $\mathbf{M}$. λ is a trade-off parameter. $\mathbf{M}$ that minimises the loss function is the desired result.

Mahalanobis distance can be obtained by the measurement matrix $\mathbf{M}$. Supposed data set $\mathbf{X} = (x_1, \dots, x_n)$ with dimension d , Mahalanobis distance between x_i and x_j is:

$$d_{\mathbf{M}}(x_i, x_j) = \sqrt{(x_i - x_j)^{\top} \mathbf{M} (x_i - x_j)}, \quad (3)$$

For linear metric learning, $\mathbf{M}$ is a $d \times d$ measure matrix. $\mathbf{M} \in S_d^+$ is a positive semi-definite matrix and can be decomposed as $\mathbf{M} = \mathbf{L}^{\top} \mathbf{L}$, where $\mathbf{L}$ is a linear transformation matrix. When $\mathbf{M} = \mathbf{I}$, Mahalanobis distance is reduced to Euclidean distance. Actually, Mahalanobis distance is a generalisation of Euclidean distance. To simplify the calculation, this work uses squared Mahalanobis distance.

Many Mahalanobis based metric learning approaches have been proposed. For example, information theoretic metric learning (ITML) [35] makes the solution close to distance metric prior

$\mathbf{M}_0$ with some given constraints satisfied. Neighborhood Components Analysis (NCA) [36] minimizes the expected leave-one-out error of k -NN classifier. Mahalanobis Metric for Clustering (MMC) [37] maximizes the distance between dissimilar samples under the constraint that the sum of distance between similar samples are smaller than a predefined threshold. Metric learning is used in online learning [38], [39], [40], [41], multi-task learning [42], [43] and unsupervised learning [44], [45], [46], etc. In Mahalanobis metric learning, large margin nearest neighbour (LMNN) [24] learns a linear feature space, where similar neighbour samples are closer and dissimilar samples are more separated. LMNN divides the k nearest neighbours of x_i into *target neighbour* and *imposter*. The k nearest neighbours of x_i with the same label x_j are called *target neighbour*, while the rest nearest neighbours with different labels x_l are called *imposter*. The sample with its *imposter* and *target neighbour* are formed into a triplet (x_i, x_j, x_l) , and the triplet set is denoted as $(i, j, l) \in \mathcal{X}^t$. In order to make similar samples closer and dissimilar samples separate apart, LMNN defines the following loss function:

$$\varepsilon(\mathbf{M}) = (1 - \mu)\varepsilon_{\text{pull}}(\mathbf{M}) + \mu\varepsilon_{\text{push}}(\mathbf{M}), \quad (4)$$

where

$$\begin{aligned} \varepsilon_{\text{pull}}(\mathbf{M}) &= \sum_{j \sim i} d_{\mathbf{M}}^2(x_i, x_j) \\ \varepsilon_{\text{push}}(\mathbf{M}) &= \sum_{(i, j, l) \in \mathcal{X}^t} (1 - \theta_{il}) \xi_{ijl} \end{aligned} \quad (5)$$

$\varepsilon_{\text{pull}}(\mathbf{M})$ reduces the distance between each sample and its *target neighbour* and $\varepsilon_{\text{push}}(\mathbf{M})$ adopts the hinge loss $\xi_{ijl} = [1 + d_{\mathbf{M}}^2(x_i, x_j) - d_{\mathbf{M}}^2(x_i, x_l)]_+$, where $[z]_+ = \max(z, 0)$, which can make *imposter* and the farthest similar neighbour sample retain at least one distance unit. Binary variable $\theta_{il} \in \{0, 1\}$ indicates that x_i and x_l are in the same class if $\theta_{il} = 1$; and in different classes otherwise. $\mu \in [0, 1]$ controls the force of pulling and pushing. By minimising the loss function, a linear projection matrix $\mathbf{L}$ can be learned. Transformed data can be obtained by projecting the data with $\mathbf{L}$. The transformed data is expected to improve the performance of k -NN classifier. Since LMNN has no regularisation, it is easy to cause over-fitting [47].

Besides, several methods have been proposed to improve LMNN. For example, parameter free large margin nearest neighbour (PFLMNN) [48] only pushes imposters separate apart to simplify the optimisation process and parameter tuning. Distance metric learning with the universum (DMLU) [49] introduces prior information to improve performance of LMNN. Some algorithms extend LMNN to learn a nonlinear distance [50], [51]. In addition, some researchers modify LMNN to adapt different scenarios. Shibin [52] et al. learns a shared Mahalanobis metric $\mathbf{M}_0$ and task-specific matrices, where $\mathbf{M}_0$ captures the general trends and task-specific matrices $\mathbf{M}_t$ make similar samples closer and dissimilar samples pushed away within task. Imbalance metric learning (IML) [53] divides samples into four types of sample pairs according to sample class, and each kind of sample pairs is assigned to a fixed weight. Distance metric by balancing KL-divergence (DMBK) [54]

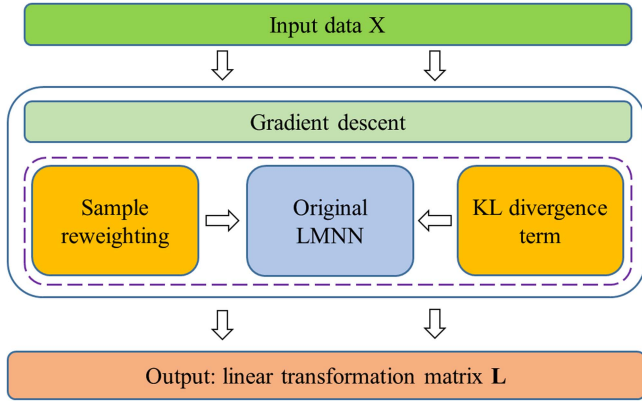


Fig. 1. The overall framework of ILMNN algorithm. The framework of ILMNN contains the construction of loss function and its optimization procedure.

minimises the modified KL divergence between classes. Iterative metric learning [55] finds a more stable neighbourhood iteratively and learns a distance metric in this subset. Although metric learning-based imbalanced algorithms can improve the performance of classifiers on imbalanced datasets, they cannot consider combining over-sampling strategies.

C. Other Methods of Imbalanced Learning

Beyond sampling methods, solutions at the algorithmic level and hybrid approaches also effectively tackle the problem of imbalanced data. Chen et al. [10] provide a comprehensive review of these solutions, spanning from traditional machine learning to deep learning techniques. At the algorithmic level, numerous researchers have proposed various innovative cost-sensitive learning schemes. In the realm of machine learning, addressing class imbalance continues to be a significant challenge, prompting several innovative solutions. For instance, Cao et al. [56] enhanced SVM performance on imbalanced datasets with their ATEC method. Chen et al. [57], [58] introduced systems like the Double-Kernelized Weighted Broad Learning System and the Class-Specific Generalized Learning System to enhance robustness and accuracy in imbalanced data scenarios. Hybrid methods are used to mix multiple techniques to comprehensively enhance the problem of classifying imbalanced data. For example, Yang et al. [59] proposed an ensemble of weighted broad networks to solve the class imbalance problem. Yang et al. [60] proposed a hybrid ensemble classifier framework combining density-based undersampling and cost-sensitive methods, significantly enhancing class imbalance handling on real-world datasets.

III. PROPOSED METHODOLOGY

In this section, we provide a comprehensive introduction to our proposed ILMNN. The overall framework of ILMNN is shown in Fig. 1. Given an imbalanced data set $\mathbf{X} \in \mathbf{R}^{n \times d}$, the goal of this work is to learn a linear feature space, where oversampling methods can work better. The feature space is parameterised by a projection matrix $\mathbf{L} \in \mathbf{R}^{d \times d}$. After applying oversampling methods to the projected imbalanced data $\mathbf{X}_{ori}^* = \mathbf{L}\mathbf{X}$, we obtain the balance data set $\mathbf{X}^* = \mathbf{X}_{ori}^* \cup$

$\mathbf{X}_{syn}^*$, where $\mathbf{X}_{syn}^*$ denotes minority-class sample set generated by oversampling. The application of classifiers on the balanced dataset $\mathbf{X}$ leads to improved classifier performance.

A. Sample Reweighting

Although LMNN has been shown to enhance the performance of the k -NN classifier, directly applying LMNN to imbalanced datasets presents a challenge due to the inter-class imbalance problem. This issue hinders the effectiveness of oversampling methods in improving performance. To address these concerns, we propose the Imbalanced Learning with Mahalanobis Neural Network (ILMNN), which introduces weighted assignment of samples. Specifically, when dealing with imbalanced datasets with a substantial disparity in the number of majority-class and minority-class samples, the loss function is dominated by the majority-class samples. Existing metric learning methods treat all samples equally, resulting in classifiers that primarily focus on the majority-class samples while suppressing the minority-class samples within the learned feature space. Consequently, subsequent oversampling methods yield unsatisfactory results, as the performance is adversely affected.

To mitigate the impact of class imbalance, we advocate for the assignment of different weights to individual samples. Smaller weights are assigned to the majority-class samples, while larger weights are assigned to the minority-class samples. Through this re-weighting process, the negative effects of class imbalance on classifier performance can be alleviated.

Fig. 2 illustrates two typical examples of assigning different weights to samples, which is dependent on local distribution. Similar samples close to overlapping area are more likely to be misclassified, leading to performance decrease of classifiers. If we assign larger weights to samples near to decision boundary, this can increase the impact on the loss function, thereby paying more attention to samples close to the decision boundary. To deal with inter-class imbalance, weights of majority-class samples are decreased. Therefore, contributions of majority-class samples are reduced to similar as those of minority-class samples.

First, we calculate k nearest neighbours of each sample (e.g., x_i) and denote as $K(x_i)$. All the neighbours of x_i which satisfy $y_i \neq y_k$ are defined as set $D(x_i)$. Then the probability that a sample belonging to other classes $Dsim(x_i)$ is calculated by

$$Dsim(x_i) = \frac{\sum_{x_d^M \in D(x_i^M)} e^{-\|x_i^M - x_d^M\|^2}}{\sum_{x_k^M \in K(x_i^M)} e^{-\|x_i^M - x_k^M\|^2}}, \quad (6)$$

where the denominator is the sum of exponential distances between each sample in $K(x_i)$ and x_i ; the numerator denotes the sum of exponential distances between each sample with different labels in $D(x_i)$ and x_i . k is a trade-off parameter. A larger k indicates weights of samples are affected by local distributions more heavily. $Dsim_{x_i} \in [0, 1]$. Generally, a larger indicates $Dsim$ that x_i is closer to samples whose label is different from x_i . From another perspective, a large $Dsim(x_i)$ indicates x_i is easy to be misclassified. To focus on samples that may be misclassified, $Dsim(x_i)$ can be utilised as the weight of x_i .

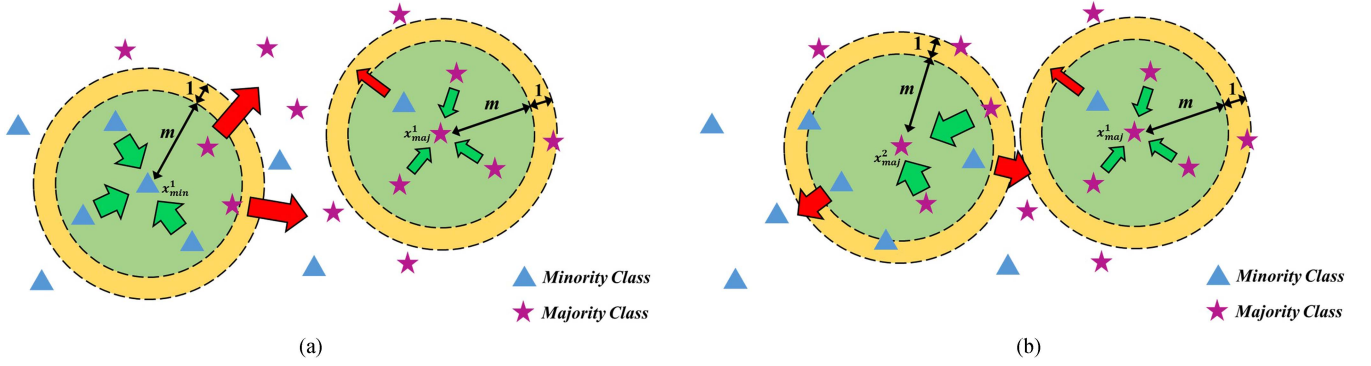


Fig. 2. Illustration of weight differences between similar samples and samples from different classes. [a] For different class of samples, ILMNN assigns higher weights to minority samples, so the pushing and pulling force acting on the minority class samples are increased. [b] For similar samples, ILMNN assigns large weights to samples closer to the decision boundary. The samples near decision boundary have large pushing and pulling force to their neighbours. The arrow indicates the size of the sample being pushed away and pulled in.

In other words, samples that exhibit closer proximity to samples from different classes should be assigned higher weights compared to those within the same class. When utilizing $Dsim(x_i)$ directly as sample weights, x_i with k nearest neighbors belonging to the same class would result in a weight of zero. Consequently, the majority of samples would have zero weights. This implies that the optimization of their distribution is disregarded, which can lead to the invasion of different labeled samples into the local distribution of unused samples, thereby worsening their local distribution. To address this issue, we introduce a constant term to ensure that the sample weight w_i is always greater than or equal to zero. Consequently, the optimization of the local distribution of all samples can be achieved. Additionally, the issue of between-class imbalance exacerbates the preference of metric learning towards the majority-class samples, while neglecting the minority-class samples. To balance the contributions of samples from different classes, we divide the weights of majority-class samples by the imbalance ratio (IR), which is defined as $\frac{|N_{maj}|}{|N_{min}|}$. This adjustment ensures that the contribution of the minority class is comparable to that of the majority class.

Each sample is subject to pulling force and pushing force by ε_{pull} and ε_{push} in formula (5). With such a weighting strategy, samples from different classes with various local distributions can have different effects on the loss function. In this setting, the loss function penalises more minority-class samples and samples that are prone to be misclassified. As a result, larger pushing and pulling forces are acted on the above two kinds of samples, which leads to that neighbours of these samples are more closely surround by these anchor samples while the margins between these anchor samples and samples with different labels increase. Meanwhile, impacts of majority-class samples on the loss function can be suppressed. The modified loss function can be given by:

$$Loss_{Weight} = \sum_{(i,j,l) \in \mathcal{X}^t} w_i \left[(1 - \theta_{il}) \xi_{ijl} + \sum_{j \sim i} d_M^2(x_i, x_j) \right], \quad (7)$$

where w_i denotes the weight of x_i . We set the x_i 's label $y_i=1$ if x_i belongs to majority class and $y_i=-1$ otherwise. w_i is defined by:

$$w_i = \begin{cases} 1 + Dsim(x_i^M) & y_i = -1 \\ \frac{|N_{min}|}{|N_{maj}|} [1 + Dsim(x_i^M)] & y_i = 1, \end{cases} \quad (8)$$

B. Addressing Imbalanced Feature Space With KL Divergence

LMNN incorporates constraints involving triplets consisting of similar and dissimilar samples. However, a drawback of LMNN is that it reduces the distance between anchor samples and neighbor samples. As a result, neighbor samples may collapse towards the anchor samples, and each sample can be treated as an anchor, leading to the formation of multiple clusters within each class. The feature space learned by LMNN exhibits an imbalanced sample distribution. Furthermore, when oversampling methods operate in such a feature space, they tend to generate multiple clusters within the minority class, thereby exacerbating the classification challenge. To address this issue, we propose a solution that constrains the sample distribution in the feature space to closely resemble the distribution of samples in the original space, where class imbalance is not as severe. Specifically, we introduce KL divergence to achieve this objective. Let P_i denote the probability distribution of sample x_i in the initial space and Q_i denote the probability distribution of the sample x_i^M in the feature space. The KL divergence for sample x_i can be expressed as follows:

$$KL(P_i \| Q_i) = \sum_{\{j|y_i=y_j\}} p_{ij} \log \frac{p_{ij}}{q_{ij}}, \quad (9)$$

where p_{ij} represents the probability that x_j is the neighbor of sample x_i , and q_{ij} denotes the probability that x_j^M is the neighbour of sample x_i^M . p_{ij} and q_{ij} are expressed by:

$$p_{ij} = \frac{e^{-\|x_i - x_j\|^2}}{\sum_{\{l|y_i=y_l\}} e^{-\|x_i - x_l\|^2}} \quad (10)$$

$$q_{ij} = \frac{e^{-\|x_i^M - x_j^M\|^2}}{\sum_{\{l|y_i=y_l\}} e^{-\|x_i^M - x_l^M\|^2}}, \quad (11)$$

In Euclidean space, the value of p_{ij} is relatively high for samples that are close to each other, while it approaches zero for samples that are far apart. Similarly, in the feature space, we can perform the same analysis for q_{ij} . The dissimilarity between the sample distribution in the feature space and the initial distribution can be quantified using KL divergence. A smaller value of KL divergence indicates that the local distributions of samples are closer. As the probability distribution in the original space remains unchanged, the first term of the KL divergence can be treated as a constant, leading to the degeneration of KL divergence into cross-entropy:

$$KL(P_i||Q_i) = \sum_{\{j|y_i=y_j\}} -p_{ij} \log q_{ij} \quad (12)$$

ILMNN adds the sum of the cross entropy over all samples to the loss function. KL divergence term in the loss function is:

$$\begin{aligned} Loss_{KL} &= \sum_i KL(P_i||Q_i) \\ &= \sum_i \sum_{\{j|y_i=y_j\}} -p_{ij} \log q_{ij}. \end{aligned} \quad (13)$$

By incorporating $Loss_{KL}$, ILMNN effectively accounts for the original distribution, thereby alleviating the issue of intra-class problems in the feature space. Furthermore, this approach preserves the local structure present in the original distribution, which proves beneficial for subsequent oversampling methods. Meanwhile, KL divergence is crucial for aligning feature space distributions with original data, effectively preserving class balance and addressing imbalances. Its sensitivity to discrepancies in probability distributions is key in preventing minority classes from being overwhelmed. Additionally, KL divergence's properties simplify its integration into optimization algorithms, enabling straightforward, gradient-based optimization in complex models.

C. The Overall Objective Function

The loss function of our model comprises two components, namely $Loss_{Weight}$ and $Loss_{KL}$. To prevent overfitting, we incorporate F-norm regularization $\|\mathbf{M}\|_F^2$ into the model. As a result, the overall loss function can be expressed as below:

$$\begin{aligned} \min \quad & \sum_{j \sim i} w_i (1 - y_{il}) [1 + d_{\mathbf{M}}^2(x_i, x_j) - d_{\mathbf{M}}^2(x_i, x_l)] \\ & + \mu_1 \sum_c \sum_{\{i,j|y_i=y_j\}} -p_{ij} \log q_{ij} \\ & + \sum_{j \sim i} w_i d_{\mathbf{M}}^2(x_i, x_j) + \mu_2 \|\mathbf{M}\|_F^2 \\ \text{s.t.} \quad & \mathbf{M} \geq 0. \end{aligned} \quad (14)$$

Through gradient descent, the loss function is minimized, and the required feature space can be obtained.

D. Optimizing the ILMNN Process

This subsection describes how to solve the loss function. In the original LMNN, positive semi-definite programming (PSD) [24] is used to optimise the original loss function. PSD first computes gradient descent of the loss function to find the next iteration $\mathbf{M}_{t+1}$ of matrix $\mathbf{M}_t$. Then $\mathbf{M}_{t+1}$ is projected back to positive semi-definite cone that satisfies positive semi-definite constraint. Since the original function is convex with respect to $\mathbf{M}$, PSD can obtain the global optimum theoretically. However, PSD suffers from high complexity. Another method is to directly performs gradient descent on the projection matrix $\mathbf{L}$. Since $\mathbf{M}$ is a positive semi-definite matrix, $\mathbf{M}$ can be decomposed into $\mathbf{M} = \mathbf{L}^T \mathbf{L}$, which converts the solution to decomposing $\mathbf{M}$ into solving the projection matrix $\mathbf{L}$, which is faster than PSD. Therefore, we directly use gradient descent to solve the projection matrix $\mathbf{L}$.

Assume $\mathbf{C}_{ij} = (\vec{x}_i - \vec{x}_j)(\vec{x}_i - \vec{x}_j)^T$, the distance between samples in the feature space can be defined as follows:

$$d_{\mathbf{M}}(\vec{x}_i, \vec{x}_j) = \text{Tr}(\mathbf{C}_{ij} \mathbf{M}). \quad (15)$$

Deriving the projection matrix $\mathbf{L}$ with respect to distances between samples, we obtain:

$$\begin{aligned} \nabla_{\mathbf{L}} d_{\mathbf{M}}(x_i, x_j) &= 2\mathbf{L}(x_i - x_j)(x_i - x_j)^T \\ &= 2\mathbf{L}\mathbf{C}_{ij}. \end{aligned} \quad (16)$$

$Loss_{Weight}$ is defined as the sum of weighted $\epsilon(\text{push})$ and $\epsilon(\text{pull})$. Substituting formula (17) into formula (7), and combining $\mathbf{M} = \mathbf{L}^T \mathbf{L}$, we can obtain the derivation:

$$\nabla_{\mathbf{L}}^{t+1} Loss_{Weight} = 2\mathbf{L}^t \sum_i w_i^t (\nabla_{\mathbf{L}} \epsilon(\text{push}) + \nabla_{\mathbf{L}} \epsilon(\text{pull})), \quad (17)$$

where

$$\begin{aligned} \nabla_{\mathbf{L}} \epsilon(\text{push}) &= \sum_{j \sim i} \mathbf{C}_{ij}, \\ \nabla_{\mathbf{L}} \epsilon(\text{pull}) &= \sum_{(i,j,l) \in \mathcal{X}^t} (\mathbf{C}_{ij} - \mathbf{C}_{il}). \end{aligned} \quad (18)$$

The derivative of KL divergence can be obtained by substituting q_{ij} into formula (13):

$$Loss_{KL} = \sum_{(i,j)} p_{ij} \left[d_{\mathbf{M}}(x_i, x_j) + \log \sum_{\{l|y_i=y_l\}} e^{d_{\mathbf{M}}(x_i, x_l)} \right]. \quad (19)$$

The cross entropy is derived as:

$$\nabla_{\mathbf{L}} p_{ij} \log q_{ij} = \frac{p_{ij}}{q_{ij}} \nabla_{\mathbf{L}} q_{ij}. \quad (20)$$

Combining Formula (21) and Formula (22), we get

$$\nabla_{\mathbf{L}}^{t+1} Loss_{KL} = 2\mathbf{L} \sum_{(i,j)} p_{ij} \left(\mathbf{C}_{ij} - \sum_{\{l|y_i=y_l\}} q_{ij} \mathbf{C}_{il} \right). \quad (21)$$

By adding Formula (18) and Formula (23), we can get the derivation formula of the loss function:

$$\nabla_{\mathbf{L}}^{t+1} Loss = \nabla_{\mathbf{L}}^{t+1} Loss_{Weight} + \nabla_{\mathbf{L}}^{t+1} Loss_{KL} + 4\mathbf{M}\mathbf{L}. \quad (22)$$

Algorithm 1: ILMNN.

Input: Training set $\mathbf{X} = (x_1, x_2, \dots, x_n)$
Parameter: Number of nearest neighbor k , trade-off parameter μ_1 and μ_2
Initialize: Transformation matrix $\mathbf{L} = \mathbf{I}$
Procedure:

- 1: **for** i in $1, \dots, N$ **do**
- 2: Calculates the distribution in original space p_{ij} by formula (11);
- 3: **end for**
- 4: **while** not converge **do**
- 5: Obtain projected data $\mathbf{X}^* = \mathbf{L}\mathbf{X}$;
- 6: **for** i in $1, \dots, N$ **do**
- 7: Obtains sample weight w_i by formula (7) and (9);
- 8: Calculates sample distribution in feature space q_{ij} by formula (12);
- 9: **end for**
- 10: Update overall objective function $Loss$ by formula (18);
- 11: Updates transform matrix $\mathbf{L}$ by formula (26);
- 12: **Until** objective function **Converge**
- 13: **end while**

Output: Transformation matrix $\mathbf{L}$.

Through the gradient descent, the solution of projection matrix $\mathbf{L}$ is obtained. Algorithm 1 summarizes the overall framework of ILMNN.

E. Time Complexity Analysis

We perform time complexity analysis for ILMNN. The overall time complexity of ILMNN T_{ILMNN} is related to five parts: data transformation, finding sample neighbours process, calculating weights w , KL divergence $KL(P||Q)$ and outer product C_{ij} . For data set which has n samples with dimension d , the cost of data transformation to feature space is $O(nd^2)$. The cost of finding sample neighbours is $O(dn^2)$. In the process of finding sample neighbour, the distance between samples is also obtained and can reuse for later procedure. The complexity of calculating sample weights is $O(n)$. The complexity of calculating KL divergence is $O(n^2)$. The complexity of outer product is $O(nd^2)$. Therefore, the complexity of ILMNN T_{ILMNN} is $O(nd^2 + dn^2 + n^2 + n + nd^2)$. Since d is much smaller than n , the computational cost of ILMNN is $O(n^2)$.

IV. EXPERIMENT

In this section, we conduct experiments by comparing our ILMNN with state-of-the-art competing methods on 16 real-world datasets. Besides, we conduct ablation study to investigate effectiveness of components in our method. Finally, we conduct statistical test to verify whether there are significant differences between our method and competing methods.

TABLE I
THE ATTRIBUTION OF DATASETS

Dataset	Samples	Feature	Majority	Minority	IR
glass_0_1_2_3_vs_4_5_6	214	9	163	51	3.2
winequality_red_4	1599	11	1546	53	29.17
ecoli4	336	7	316	20	15.8
ecoli_0_1_4_7_vs_5_6	332	6	307	25	12.28
ecoli3	336	7	301	35	8.6
yeast4	1484	8	1433	51	28.1
yeast_0_2_5_6_vs_3_7_8_9	1004	8	905	99	9.14
yeast5	1484	8	1440	44	32.73
yeast_1_4_5_8_vs_7	693	8	663	30	22.1
yeast_1_vs_7	459	7	429	30	14.3
climate	540	18	494	46	10.74
ozone_level	2536	72	2462	74	33.74
thoracic_surgery	470	16	400	70	5.71
yeast_me2	1484	8	1433	51	28.1
newthyroid2	215	56	180	35	5.14
seismic_bumps	2584	18	2414	170	14.2

A. Datasets & Evaluation Measure

We test the performance of our ILMNN on 16 datasets with various sizes, dimensions, and imbalanced ratios. These datasets are from KEEL [61] and UCI [62]. We divide datasets with more than two classes into majority and the minority class by merging classes. Table I provides attributes of datasets, including the total number of samples, the number of samples in each class, the number of features and imbalance ratio (IR). To make different features numerically comparable, those datasets are first normalised to [0,1] and then input to metric learning algorithm to obtain the projection matrix $\mathbf{L}$. The training data is linearly transformed by $x^* = \mathbf{L}x$, and oversampling is performed in the transformed space to form a balanced datasets, on which the classifier is trained. Finally, the test data is projected into feature space with $\mathbf{L}$ and then classified with the learned classifier. Experiments are conducted on Intel Core i7-8700K CPU@3.70 GHz with 16 GB RAM. We adopt five-fold cross-validation and report the average along with corresponding variance.

The majority class samples dominate the overall accuracy in an imbalanced issue setting because there are more samples from the majority class than from the minority class. Therefore, the accuracy is not suitable as a criterion. G-means comprehensively considers the classification performance of the majority class and the minority class, which is defined as follow:

$$G - means = \sqrt{\frac{TP}{TP + FN} \times \frac{TN}{TN + FP}}. \quad (23)$$

where TP and TN indicate correct predictions of positive and negative classes, respectively. FN and FP represent errors where the model incorrectly predicts negative when it's positive and positive when it's negative, respectively.

In addition, AUC [63] is another evaluation to measure the classification performance in imbalanced data, which denotes the area under ROC curve that integrates Precision and Recall. The closer the AUC is to 1, the better the performance of the classifier is. AUC and G-means are utilised as evaluation indicators in the following experiment results. The calculation of the G-means and AUC is based on the confusion matrix.

TABLE II
COMPARISONS BETWEEN EUCLIDEAN SPACE, LMNN, ITML, IML AND ILMNN WHEN SMOTE ADOPTS k -NN AND NAIVE BAYES CLASSIFIER IN TERMS OF AUC

	Euclidean Space		LMNN		ITML		IML		ILMNN	
	KNN	Naive Bayes	KNN	Naive Bayes	KNN	Naive Bayes	KNN	Naive Bayes	KNN	Naive Bayes
glass_0_1_2_3_vs_4_5_6	0.914±0.026	0.700±0.031	0.902±0.027	0.841±0.033	0.867±0.038	0.752±0.039	0.917±0.020	0.706±0.032	0.918±0.025	0.814±0.046
winequality_red_4	0.564±0.027	0.655±0.011	0.541±0.024	0.658±0.025	0.563±0.035	0.666±0.023	0.564±0.021	0.656±0.018	0.571±0.037	0.664±0.014
ecoli4	0.906±0.032	0.897±0.020	0.910±0.025	0.894±0.028	0.924±0.034	0.921±0.029	0.908±0.027	0.908±0.034	0.931±0.026	0.915±0.018
ecoli_0_1_4_7_vs_5_6	0.863±0.042	0.814±0.066	0.853±0.049	0.834±0.068	0.871±0.033	0.865±0.051	0.862±0.043	0.823±0.057	0.881±0.032	0.830±0.059
ecoli3	0.824±0.021	0.874±0.013	0.825±0.033	0.829±0.037	0.829±0.031	0.851±0.017	0.825±0.025	0.873±0.014	0.823±0.034	0.888±0.012
yeast4	0.699±0.030	0.625±0.024	0.691±0.039	0.710±0.033	0.714±0.035	0.673±0.026	0.699±0.038	0.688±0.032	0.722±0.038	0.788±0.012
yeast_0_2_5_6_vs_3_7_8_9	0.737±0.029	0.754±0.013	0.736±0.026	0.759±0.028	0.738±0.023	0.733±0.016	0.737±0.021	0.754±0.014	0.742±0.024	0.792±0.012
yeast5	0.838±0.048	0.868±0.018	0.844±0.033	0.899±0.027	0.855±0.043	0.881±0.039	0.839±0.044	0.924±0.017	0.871±0.044	0.937±0.027
yeast_1_4_5_8_vs_7	0.569±0.041	0.563±0.034	0.574±0.037	0.587±0.043	0.552±0.030	0.592±0.036	0.567±0.041	0.589±0.027	0.595±0.042	0.612±0.038
yeast_1_vs_7	0.690±0.037	0.621±0.017	0.682±0.034	0.684±0.032	0.683±0.047	0.667±0.033	0.689±0.039	0.644±0.025	0.711±0.021	0.704±0.044
climate	0.684±0.037	0.787±0.024	0.733±0.019	0.770±0.025	0.724±0.024	0.779±0.026	0.694±0.036	0.792±0.023	0.777±0.029	0.768±0.030
ozone_level	0.708±0.017	0.751±0.017	0.688±0.013	0.773±0.016	0.710±0.025	0.745±0.017	0.715±0.01	0.757±0.014	0.716±0.027	0.773±0.019
thoracic_surgery	0.513±0.032	0.501±0.010	0.508±0.033	0.540±0.024	0.512±0.037	0.502±0.034	0.514±0.034	0.523±0.018	0.519±0.048	0.516±0.01
yeast_me2	0.694±0.038	0.619±0.025	0.690±0.031	0.693±0.014	0.696±0.048	0.671±0.023	0.693±0.039	0.665±0.028	0.712±0.032	0.783±0.026
newthyroid2	0.943±0.021	0.909±0.03	0.944±0.026	0.965±0.018	0.938±0.027	0.918±0.043	0.942±0.024	0.918±0.036	0.943±0.022	0.968±0.016
seismic_bumps	0.597±0.029	0.596±0.014	0.590±0.018	0.663±0.013	0.578±0.025	0.652±0.034	0.596±0.022	0.598±0.024	0.586±0.023	0.681±0.016
Average AUC	0.734	0.721	0.732	0.756	0.735	0.742	0.735	0.739	0.751	0.777
Average Rank	3.3125	4.4375	3.8125	2.625	3.1875	3.0625	3.1875	3.1875	1.5	1.6875

The bolded text indicates the optimal result of the model.

B. Parameter Setting of All the Methods

For a fair comparison, we provide the detailed parameter settings of all the competing methods and our method for reproducing the experimental results. We compare ILMNN with three metric learning-based algorithms, namely, LMNN [24], IML [53] and ITML [35]. The parameter of three metric learning are as followed:

- 1) LMNN: A classical metric learning that the k -nearest neighbours of same class are closer than those of other class. To avoid over-fitting, regularisation is imposed to LMNN. The number of neighbours $k = 5$; trade-off parameter $\mu_1 = 0.5$; regularisation parameter $\mu_2 = 0.5$.
- 2) ITML: A classical metric learning that learns a feature space which is closer to initial space while given constraints are satisfied. The number of constraints to generate $constrain = 80$, $\gamma = 1$.
- 3) IML: A variant of LMNN on dealing with imbalanced data. The number of neighbours $k=5$, the weight $a=b=0.5$, margin parameter $m = 100$, regularisation $\lambda = 0.1$.

Besides, we adopt three classical oversampling, namely, SMOTE [28], ADASYN [29] and MWMOTE [30], in the feature space which is constructed by metric learning. The parameter of three oversampling are as followed:

- 1) SMOTE: A classical oversampling approach that interpolates in minority classes to generate samples. We set the number of neighbours is $k = 5$.
- 2) ADASYN: A variant of SMOTE approach that assigns different sampling weights to minority class samples. We set the number of neighbours is $k = 5$.

- 3) MWMOTE: A variant of SMOTE approach that combines sampling weighting and clustering approaches. The number of neighbours for constructing noisy minority class samples is $k_1 = 5$, The number of majority neighbours for constructing informative minority set is $k_2 = 5$, The number of minority neighbours for constructing informative minority set is $k_3 = 5$.

For our ILMNN, the number of neighbours is $k = 5$, the KL divergence is set to $\mu_1=10$ and the regularisation parameter is $\mu_2 = 0.5$. When calculating the sample weights w , ILMNN considers four nearest neighbours for each sample. As comparison, the result of oversampling directly in Euclidean space are also given. In addition, the average value and average ranking of AUC and G-means are reported in Tables II, III, IV, and V.

C. Comparisons With Metric Learning-Based Methods

We study effects of feature spaces learnt by our ILMNN by comparing with feature spaces learnt by LMNN, IML and ITML along with Euclidean space, where the over-sampling method SMOTE adopts two typical classifiers k -NN (with $k=3$) and Gaussian naive Bayes classifier. Tables II–III show comparisons of using k -NN classifier and Gaussian naive Bayes classifier in terms of AUC and G-means respectively. We have the following observation:

- 1) LMNN does not appear obvious performance improvement in terms of the average AUC, G-means and rankings, compared to SMOTE working on Euclidean space with k -NN classifier. Similarly, performances of ITML and IML are similar to performance of LMNN. In other words, LMNN, ITML and IML failed to improve the performance

TABLE III
COMPARISONS BETWEEN EUCLIDEAN SPACE, LMNN, ITML, IML AND ILMNN WHEN SMOTE ADOPTS k -NN AND NAIVE BAYES CLASSIFIER IN TERMS OF G-MEANS

	Euclidean Space		LMNN		ITML		IML		ILMNN	
	KNN	Naïve Bayes	KNN	Naïve Bayes	KNN	Naïve Bayes	KNN	Naïve Bayes	KNN	Naïve Bayes
glass_0_1_2_3_vs_4_5_6	0.911 ± 0.025	0.614 ± 0.046	0.897 ± 0.023	0.813 ± 0.047	0.852 ± 0.052	0.681 ± 0.057	0.916 ± 0.029	0.621 ± 0.045	0.915 ± 0.020	0.774 ± 0.063
winequality_red_4	0.435 ± 0.068	0.643 ± 0.028	0.380 ± 0.064	0.643 ± 0.038	0.459 ± 0.05	0.646 ± 0.039	0.435 ± 0.065	0.644 ± 0.026	0.457 ± 0.067	0.657 ± 0.020
ecoli4	0.895 ± 0.032	0.890 ± 0.035	0.902 ± 0.026	0.884 ± 0.028	0.916 ± 0.031	0.916 ± 0.024	0.898 ± 0.0387	0.900 ± 0.042	0.924 ± 0.021	0.911 ± 0.015
ecoli_0_1_4_7_vs_5_6	0.795 ± 0.084	0.714 ± 0.122	0.820 ± 0.0657	0.765 ± 0.102	0.849 ± 0.045	0.829 ± 0.071	0.794 ± 0.072	0.735 ± 0.102	0.871 ± 0.034	0.759 ± 0.107
ecoli3	0.810 ± 0.031	0.870 ± 0.025	0.811 ± 0.043	0.814 ± 0.049	0.817 ± 0.032	0.845 ± 0.015	0.812 ± 0.037	0.869 ± 0.026	0.810 ± 0.046	0.886 ± 0.012
yeast4	0.639 ± 0.048	0.548 ± 0.036	0.627 ± 0.053	0.682 ± 0.059	0.666 ± 0.045	0.618 ± 0.057	0.639 ± 0.040	0.657 ± 0.043	0.678 ± 0.059	0.780 ± 0.010
yeast_0_2_5_6_vs_3_7_8_9	0.714 ± 0.032	0.723 ± 0.028	0.712 ± 0.036	0.732 ± 0.021	0.717 ± 0.023	0.694 ± 0.035	0.714 ± 0.034	0.724 ± 0.025	0.721 ± 0.037	0.782 ± 0.023
yeast5	0.813 ± 0.051	0.859 ± 0.015	0.824 ± 0.047	0.891 ± 0.023	0.838 ± 0.056	0.869 ± 0.047	0.815 ± 0.053	0.922 ± 0.017	0.857 ± 0.052	0.935 ± 0.027
yeast_1_4_5_8_vs_7	0.428 ± 0.074	0.429 ± 0.035	0.445 ± 0.076	0.542 ± 0.068	0.405 ± 0.063	0.539 ± 0.053	0.417 ± 0.074	0.502 ± 0.037	0.482 ± 0.092	0.572 ± 0.029
yeast_1_vs_7	0.648 ± 0.053	0.561 ± 0.027	0.632 ± 0.068	0.647 ± 0.064	0.647 ± 0.058	0.611 ± 0.063	0.647 ± 0.052	0.603 ± 0.035	0.681 ± 0.021	0.677 ± 0.054
climate	0.675 ± 0.031	0.769 ± 0.026	0.723 ± 0.018	0.753 ± 0.037	0.717 ± 0.039	0.764 ± 0.037	0.686 ± 0.038	0.776 ± 0.031	0.768 ± 0.034	0.677 ± 0.058
ozone_level	0.678 ± 0.028	0.747 ± 0.013	0.634 ± 0.023	0.768 ± 0.017	0.684 ± 0.038	0.738 ± 0.015	0.685 ± 0.025	0.753 ± 0.013	0.644 ± 0.027	0.769 ± 0.016
thoracic_surgery	0.437 ± 0.054	0.225 ± 0.022	0.423 ± 0.064	0.528 ± 0.027	0.463 ± 0.036	0.255 ± 0.117	0.441 ± 0.041	0.346 ± 0.049	0.440 ± 0.053	0.354 ± 0.045
yeast_me2	0.629 ± 0.046	0.544 ± 0.023	0.628 ± 0.041	0.664 ± 0.029	0.640 ± 0.054	0.616 ± 0.047	0.627 ± 0.040	0.628 ± 0.047	0.667 ± 0.048	0.773 ± 0.037
newthyroid2	0.938 ± 0.034	0.900 ± 0.037	0.939 ± 0.025	0.963 ± 0.018	0.933 ± 0.026	0.907 ± 0.058	0.937 ± 0.038	0.910 ± 0.035	0.938 ± 0.028	0.966 ± 0.016
seismic_bumps	0.554 ± 0.035	0.552 ± 0.027	0.545 ± 0.028	0.655 ± 0.014	0.518 ± 0.047	0.615 ± 0.052	0.553 ± 0.031	0.569 ± 0.035	0.556 ± 0.023	0.672 ± 0.014
Average G-means	0.687	0.662	0.683	0.734	0.695	0.697	0.688	0.697	0.713	0.751
Average Rank	3.4375	4.4375	3.6875	2.5625	2.75	3.375	3.4375	3	1.6875	1.625

The bolded text indicates the optimal result of the model.

TABLE IV
COMPARISONS BETWEEN EUCLIDEAN SPACE, LMNN, ITML, IML AND ILMNN WITH k -NN CLASSIFIER ON ADASYN IN TERMS OF AUC

	Euclidean Space	LMNN	ITML	IML	ILMNN
glass_0_1_2_3_vs_4_5_6	0.912±0.024	0.898±0.026	0.847±0.043	0.910±0.026	0.908±0.029
winequality_red_4	0.565±0.024	0.543±0.016	0.568±0.037	0.565±0.025	0.583±0.046
ecoli4	0.910±0.023	0.908±0.028	0.919±0.021	0.907±0.029	0.927±0.028
ecoli_0_1_4_7_vs_5_6	0.860±0.049	0.853±0.043	0.863±0.037	0.860±0.043	0.873±0.037
ecoli3	0.825±0.018	0.829±0.032	0.830±0.038	0.826±0.023	0.828±0.037
yeast4	0.718±0.033	0.705±0.023	0.726±0.038	0.718±0.031	0.729±0.036
yeast_0_2_5_6_vs_3_7_8_9	0.730±0.029	0.728±0.023	0.731±0.016	0.731±0.026	0.736±0.021
yeast5	0.854±0.045	0.856±0.037	0.865±0.040	0.850±0.042	0.882±0.039
yeast_1_4_5_8_vs_7	0.570±0.038	0.567±0.049	0.556±0.035	0.570±0.039	0.597±0.040
yeast_1_vs_7	0.689±0.032	0.684±0.045	0.675±0.046	0.688±0.038	0.706±0.032
climate	0.684±0.025	0.736±0.023	0.725±0.039	0.705±0.025	0.780±0.036
ozone_level	0.716±0.018	0.699±0.028	0.722±0.024	0.719±0.018	0.722±0.022
thoracic_surgery	0.516±0.031	0.509±0.036	0.520±0.027	0.514±0.035	0.523±0.046
yeast_me2	0.699±0.037	0.701±0.032	0.716±0.045	0.699±0.038	0.728±0.038
newthyroid2	0.945±0.023	0.948±0.020	0.937±0.024	0.943±0.023	0.941±0.024
seismic_bumps	0.601±0.029	0.59±0.027	0.585±0.025	0.599±0.026	0.595±0.013
Average AUC	0.737	0.735	0.737	0.738	0.754
Average Rank	3.03125	3.875	3.0625	3.46875	1.5625

The bolded text indicates the optimal result of the model.

of SMOTE with k NN classifier on imbalance datasets. The reason is that these three methods optimise sample distribution from the perspective of classifier, rather than over-sampling methods. Consequently, the sample distribution in the feature space cannot improve the performance of oversampling.

- 2) Performances of LMNN are better than those obtained in Euclidean space with k -NN classifier in terms of on

average AUC and average G-means, respectively. Performances of both ITML and IML are better than that of Euclidean space. Meanwhile, both ITML and IML methods perform worse than LMNN..

- 3) Our proposed ILMNN with k -NN classifier has achieved the best performances on 13 datasets in terms of AUC and the best performances on 11 datasets in terms of G-means. It indicates ILMNN is suitable for k -NN classifier on

TABLE V
COMPARISONS BETWEEN EUCLIDEAN SPACE, LMNN, ITML, IML AND ILMNN WITH k -NN CLASSIFIER ON ADASYN IN TERMS OF G-MEANS

	Euclidean Space	LMNN	ITML	IML	ILMNN
glass_0_1_2_3_vs_4_5_6	0.909±0.027	0.891±0.032	0.826±0.056	0.908±0.028	0.905±0.024
winequality_red_4	0.450±0.053	0.401±0.056	0.478±0.058	0.450±0.053	0.496±0.076
ecoli4	0.900±0.038	0.900±0.032	0.911±0.037	0.898±0.033	0.919±0.032
ecoli_0_1_4_7_vs_5_6	0.792±0.074	0.820±0.062	0.836±0.058	0.792±0.073	0.862±0.041
ecoli3	0.812±0.024	0.817±0.045	0.821±0.046	0.814±0.024	0.817±0.037
yeast4	0.672±0.04	0.651±0.036	0.686±0.032	0.671±0.047	0.693±0.055
yeast_0_2_5_6_vs_3_7_8_9	0.710±0.036	0.705±0.032	0.713±0.025	0.711±0.033	0.719±0.025
yeast5	0.835±0.056	0.839±0.044	0.852±0.043	0.831±0.042	0.870±0.046
yeast_1_4_5_8_vs_7	0.442±0.071	0.441±0.064	0.427±0.086	0.442±0.073	0.502±0.085
yeast_1_vs_7	0.647±0.065	0.641±0.064	0.644±0.052	0.645±0.069	0.681±0.042
climate	0.677±0.025	0.729±0.025	0.719±0.036	0.698±0.021	0.774±0.034
ozone_level	0.693±0.026	0.651±0.021	0.702±0.025	0.694±0.027	0.695±0.024
thoracic_surgery	0.445±0.056	0.434±0.051	0.477±0.026	0.444±0.048	0.449±0.063
yeast_me2	0.639±0.044	0.645±0.047	0.672±0.052	0.639±0.045	0.694±0.046
newthyroid2	0.940±0.024	0.943±0.039	0.931±0.026	0.938±0.034	0.935±0.035
seismic_bumps	0.567±0.038	0.554±0.023	0.533±0.036	0.565±0.028	0.576±0.015
Average G-means	0.695	0.692	0.702	0.696	0.724
Average Rank	3.3125	3.75	2.75	3.625	1.5625

The bolded text indicates the optimal result of the model.

TABLE VI
COMPARISONS BETWEEN EUCLIDEAN SPACE, LMNN, ITML, IML AND ILMNN WITH k -NN CLASSIFIER ON MWMOTE IN TERMS OF AUC

	Euclidean Space	LMNN	ITML	IML	ILMNN
glass_0_1_2_3_vs_4_5_6	0.905±0.026	0.902±0.034	0.861±0.033	0.914±0.024	0.916±0.027
winequality_red_4	0.577±0.032	0.560±0.024	0.575±0.043	0.581±0.035	0.584±0.043
ecoli4	0.908±0.025	0.907±0.025	0.926±0.022	0.908±0.023	0.927±0.024
ecoli_0_1_4_7_vs_5_6	0.864±0.046	0.849±0.043	0.868±0.036	0.865±0.044	0.874±0.036
ecoli3	0.819±0.034	0.824±0.034	0.835±0.037	0.827±0.033	0.829±0.035
yeast4	0.734±0.032	0.713±0.031	0.743±0.039	0.732±0.037	0.765±0.036
yeast_0_2_5_6_vs_3_7_8_9	0.738±0.025	0.728±0.015	0.738±0.024	0.743±0.023	0.736±0.026
yeast5	0.849±0.045	0.865±0.032	0.875±0.046	0.851±0.043	0.907±0.035
yeast_1_4_5_8_vs_7	0.561±0.056	0.561±0.043	0.553±0.035	0.551±0.048	0.577±0.032
yeast_1_vs_7	0.694±0.041	0.689±0.045	0.686±0.037	0.695±0.046	0.717±0.032
climate	0.698±0.024	0.741±0.026	0.733±0.035	0.721±0.024	0.790±0.027
ozone_level	0.740±0.023	0.711±0.025	0.745±0.027	0.739±0.023	0.747±0.012
thoracic_surgery	0.524±0.035	0.509±0.037	0.524±0.036	0.529±0.023	0.525±0.032
yeast_me2	0.724±0.024	0.716±0.025	0.729±0.036	0.719±0.024	0.753±0.031
newthyroid2	0.948±0.024	0.945±0.025	0.935±0.021	0.946±0.028	0.944±0.023
seismic_bumps	0.608±0.016	0.622±0.026	0.606±0.023	0.607±0.027	0.623±0.012
Average AUC	0.743	0.74	0.746	0.745	0.763
Average Rank	3.3125	4	3.125	3.0625	1.5

The bolded text indicates the optimal result of the model.

imbalance data, since it assigns different weights to samples which are near to decision boundaries or easily misclassified. Besides, ILMNN performs best on 10 datasets in terms of G-means. It indicates that ILMNN is suitable for Naive Bayes classifier. Overall, ILMNN has obtained the best performance in the average value and ranking of AUC and G-means. This verifies that our ILMNN can learn a better feature space which is more suitable over-sampling methods with different classifiers.

D. Comparisons With Different Oversampling Methods

We compare ILMNN with LMNN, ITML and IML when adopting different oversampling methods. ADASYN and MWMOTE are two classical over-sampling methods working in the feature space learned by metric learning. In ADASYN, the nearest neighbour parameter k is set to 5. In MWMOTE, the number of neighbours for constructing noisy

minority class samples k_1 , the number of majority neighbours for constructing informative minority set k_2 and the number of minority neighbours for constructing informative minority set k_3 are set to five. The parametric setting of metric learning methods are set as previously mentioned. k NN(with $k=3$) classifier is applied as basis classifier. Tables IV–VII show results of applying ADASYN and MWMOTE in terms of AUC and G-means.

From Tables IV, V, VI, and VII, ILMNN obtained better results than LMNN and IML in most datasets. Particularly, when adopting ADASYN, ILMNN outperforms LMNN, ITML and IML by 1.5% in terms of average AUC, as shown in Table IV. ILMNN outperforms other metric learning methods by over 2% in terms of average G-means, as shown in Table V. Meanwhile, ILMNN performs better than other metric learning methods when applying MWMOTE, as shown in Tables VI and VII. From Tables II and III, one can notice that ILMNN with SMOTE has achieved the highest average value and ranking of AUC and G-means. These experiments results indicate that our proposed

TABLE VII
COMPARISONS BETWEEN EUCLIDEAN SPACE, LMNN, ITML, IML AND ILMNN WITH k -NN CLASSIFIER ON MWMOTE IN TERMS OF G-MEANS

	Euclidean Space	LMNN	ITML	IML	ILMNN
glass_0_1_2_3_vs_4_5_6	0.902±0.023	0.897±0.037	0.843±0.054	0.912±0.022	0.912±0.024
winequality_red_4	0.478±0.057	0.454±0.048	0.491±0.071	0.478±0.064	0.507±0.078
ecoli4	0.898±0.034	0.897±0.035	0.918±0.032	0.898±0.031	0.920±0.021
ecoli_0_1_4_7_vs_5_6	0.795±0.087	0.817±0.064	0.842±0.056	0.799±0.071	0.863±0.035
ecoli3	0.805±0.036	0.810±0.034	0.826±0.031	0.816±0.038	0.816±0.042
yeast4	0.697±0.039	0.663±0.045	0.709±0.046	0.693±0.046	0.744±0.042
yeast_0_2_5_6_vs_3_7_8_9	0.720±0.031	0.708±0.024	0.723±0.027	0.726±0.024	0.723±0.026
yeast5	0.829±0.054	0.850±0.042	0.862±0.051	0.831±0.043	0.898±0.032
yeast_1_4_5_8_vs_7	0.403±0.095	0.413±0.093	0.396±0.061	0.389±0.081	0.459±0.080
yeast_1_vs_7	0.654±0.064	0.658±0.072	0.655±0.047	0.656±0.050	0.693±0.032
climate	0.690±0.021	0.732±0.020	0.726±0.049	0.714±0.023	0.785±0.021
ozone_level	0.723±0.024	0.672±0.039	0.733±0.027	0.721±0.025	0.729±0.024
thoracic_surgery	0.443±0.049	0.414±0.064	0.478±0.046	0.456±0.031	0.443±0.053
yeast_me2	0.676±0.036	0.671±0.036	0.690±0.058	0.671±0.037	0.731±0.041
newthyroid2	0.943±0.023	0.940±0.027	0.930±0.025	0.942±0.028	0.938±0.039
seismic_bumps	0.558±0.021	0.582±0.023	0.558±0.035	0.556±0.028	0.603±0.021
Average G-means	0.704	0.698	0.711	0.703	0.735
Average Rank	3.5	3.875	2.625	3.5	1.5

The bolded text indicates the optimal result of the model.

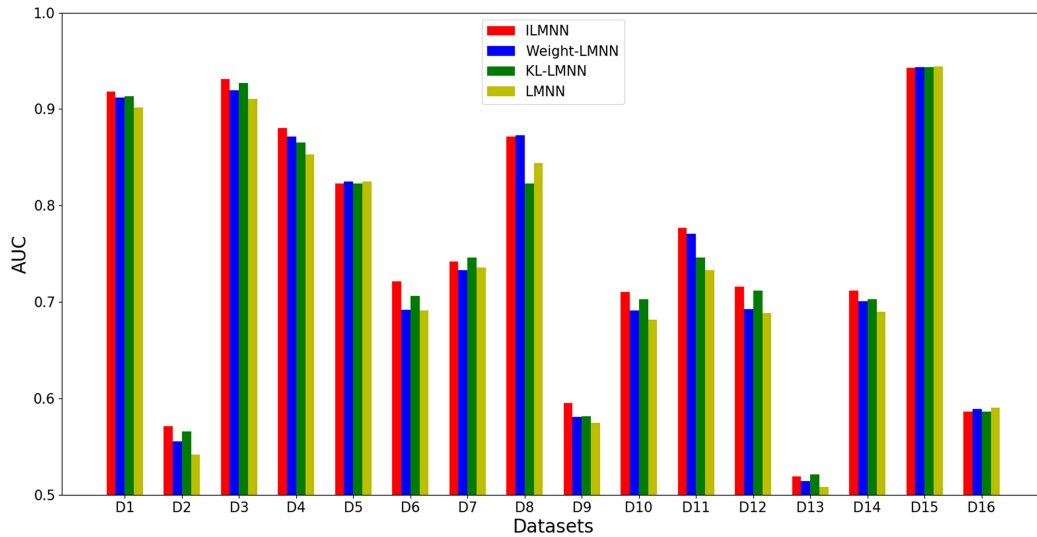


Fig. 3. Ablation study between LMNN, KL-LMNN, Weight-LMNN, and our proposed ILMNN in terms of AUC.

ILMNN can be beneficial to improving performance of different over-sampling methods in dealing with class imbalanced datasets. The reason is that ILMNN learns a better feature space by introducing KL divergence and sample re-weighting. In addition, ILMNN's reweighting strategy allocates more attention towards critical samples (e.g., samples near boundaries or samples that are prone to misclassification), which helps to improve the model's ability to recognise samples from a small number of classes.

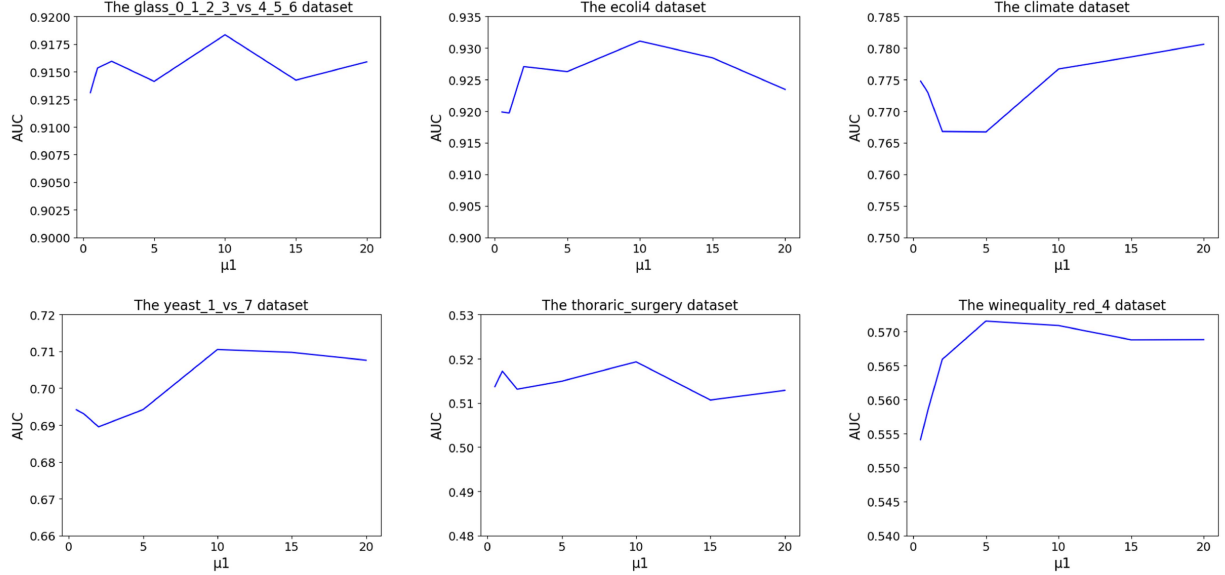
E. Ablation Study

We perform ablation study to investigate the effects of components in our proposed ILMNN, which includes re-weighting processing and KL divergence. Specifically, we compare our ILMNN with KL-LMNN and Weight-LMNN. KL-LMNN introduces KL divergence with parameter $\mu=10$ and disables

re-weighting processing, while Weight-LMNN assigns different weights to samples and disables KL-divergence. We rename the 16 datasets in Table I from D1 to D16 successively. The classifier is set to k -NN (with $k=3$). SMOTE is applied as oversampling method. The results of LMNN, KL-LMNN, Weight-LMNN and ILMNN in terms of AUC are showed in Fig. 3. We can observe that both Weight-LMNN and KL-LMNN achieve better performance on most datasets, compared with LMNN, which indicates that re-weighting or adding KL divergence can improve the performance of k -NN classifier. It is worth noting that ILMNN obtains the best performance among four algorithms. The possible reason is that re-weighting and KL divergence can effectively improve the performance of LMNN by solving inter-class and intra-class imbalance problems caused by LMNN. As a result, the utilising of KL-divergence and re-weighting processing makes our proposed ILMNN more suitable for dealing with imbalanced data classification.

TABLE VIII
 p -VALUES (ALIGNED FRIEDMAN)

algorithm	unadjusted p	p_{Bonf}	p_{Holm}	p_{Hoch}	p_{Homm}
LMNN	4.832E-6	1.933E-5	1.933E-5	1.933E-5	1.933E-5
Euclidean Space	2.116E-4	8.464E-4	6.348E-4	6.348E-4	5.89E-4
IML	3.926E-4	0.00157	7.853E-4	7.853E-4	7.853E-4
ITML	0.00177	0.00707	0.00177	0.00177	0.00177

Fig. 4. Effect of μ_1 on performance of ILMNN on glass_0_1_2_3_vs_4_5_6, ecoli4, climate, yeast_1_vs_7, thoracic_surgery and winequality_red_4.TABLE IX
AVERAGE RANKINGS OF THE ALGORITHMS (FRIEDMAN)

Algorithm	Ranking
Euclidean Space	3.31
LMNN	3.81
ITML	3.19
IML	3.19
ILMNN	1.5

F. Effects of Parameter μ_1

In this part, we investigate effects of parameters on performance of our ILMNN. In ILMNN, μ_1 controls the relative importance of KL divergence. To show the effect of μ_1 , AUC results of ILMNN with different μ_1 values are given by changing its value in the set of $\mu_1 = [0, 0.5, 1, 2, 5, 10, 15, 20]$. We conduct experiments on six datasets: glass_0_1_2_3_vs_4_5_6, ecoli4, climate, yeast_1_vs_7, thoracic_surgery and winequality_red_4.

The effects of μ_1 on performance of ILMNN are shown in Fig. 4. It can be observed that when μ_1 is set to 10, ILMNN achieves better results on datasets. A small μ_1 reduces the contribution of KL divergence. As a result, performances are mainly affected by original LMNN and re-weighting processing. When μ_1 gradually close to 10, the influence of KL divergence rises, achieving the purpose of optimising the local distribution of samples. A large μ_1 makes the KL divergence dominate the loss function, thereby ignoring the effects of other components in ILMNN. Consequently, a large μ_1 makes the learned feature space worse. In other words, we should carefully fine-tune the

value of μ_1 to balance contributions of Weighted-LMNN and KL divergence. The recommended value is $\mu_1 = 10$.

G. Nonparametric Tests

This subsection shows non-parametric statistical tests to evaluate whether ILMNN is significantly different from SMOTE, LMNN, ITML and IML. Friedman test is a well-known non-parametric test [64] for evaluating differences between algorithms concerning average ranking. Denotes the matrix $V = v_{ij}$, where v_{ij} stands for the value of average AUC in terms of i th algorithm on j th dataset, with $i \in [1, 2, \dots, I]$ and $j \in [1, 2, \dots, J]$. A statistic Ω is calculated as follows:

$$\Omega = \frac{12J}{I(I+1)} \left[\sum_i \bar{V}_i^2 - \frac{I(I+1)^2}{4} \right] \quad (24)$$

$$\bar{V}_i = \frac{1}{J} \sum_{j=1}^J V_{ij} \quad (25)$$

The value of Ω can be calculated by a chi-square χ^2 distribution when I and J are large. Therefore, the p -value is:

$$p = P\left(\chi_{a[I-1]}^2 \geq \Omega\right) \quad (26)$$

where we set the significance level α to 0.05. The results of non-parametric test are showed in Table X to Table XII.

The results of p -value and adjust p -value are given in Table VIII. The tests include Bonferroni-Dunn test, Holm test,

Hochberg test and Hommel test. Table IX shows that the average ranking of ILMNN is better than the other competing algorithms. Based on results in Tables VIII and IX, ILMNN is significantly superior to the others algorithms.

V. CONCLUSION AND FUTURE WORK

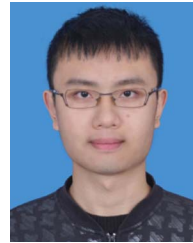
This paper proposes an imbalance metric learning method ILMNN. ILMNN construct a better feature space for improving the performance of oversampling and classification results for imbalanced data. To solve the imbalanced problem in metric learning, ILMNN first assigns large weights to minority class samples and samples which is close to the decision boundary. ILMNN then introduces KL divergence to constrain the distribution of data in the feature space closer to that in the original distribution, which alleviates intra-class imbalance problem. Experiments on 16 real-world datasets show that feature space constructed by ILMNN is better than the original space and the feature space learned by compared metric learning algorithms. In addition, non-parametric statistical tests show that ILMNN is better than compared metric learning algorithms.

Future work shall includes the following parts: 1) Considering combining other distance metrics like Wasserstein distance to obtain a better feature space. This helps to achieve a more detailed and efficient spatial representation of demand.; 2) Designing a novel oversampling method for ILMNN; 3) Combining ensemble learning with ILMNN to deal with imbalance data classification.

REFERENCES

- [1] L. Zhu, C. Lu, Z. Y. Dong, and C. Hong, "Imbalance learning machine-based power system short-term voltage stability assessment," *IEEE Trans. Ind. Informat.*, vol. 13, no. 5, pp. 2533–2543, Oct. 2017.
- [2] M. Bach, A. Werner, J. Żywiec, and W. Pluskiewicz, "The study of under and over-sampling methods' utility in analysis of highly imbalanced data on osteoporosis," *Inf. Sci.*, vol. 384, pp. 174–190, 2017.
- [3] K. Chen, W. Lei, S. Zhao, W.-S. Zheng, and R. Wang, "PCCT: Progressive class-center triplet loss for imbalanced medical image classification," *IEEE J. Biomed. Health Informat.*, vol. 27, no. 4, pp. 2026–2036, Apr. 2023.
- [4] K. E. Bennin, J. Keung, P. Phannachitta, A. Monden, and S. Mensah, "Mahakil: Diversity based oversampling approach to alleviate the class imbalance issue in software defect prediction," *IEEE Trans. Softw. Eng.*, vol. 44, no. 6, pp. 534–550, Jun. 2018.
- [5] Q. Song, Y. Guo, and M. Shepperd, "A comprehensive investigation of the role of imbalanced learning for software defect prediction," *IEEE Trans. Softw. Eng.*, vol. 45, no. 12, pp. 1253–1269, Dec. 2019.
- [6] W. Chen, K. Yang, Y. Shi, Q. Feng, C. Zhang, and Z. Yu, "Kernel-based class-specific broad learning system for software defect prediction," in *Proc. 8th Int. Conf. Cybern. Comput. Social Syst.*, 2021, pp. 109–114.
- [7] X. Wan, Z. Zheng, and Y. Liu, "SPE²: Self-paced ensemble of ensembles for software defect prediction," *IEEE Trans. Rel.*, vol. 71, no. 2, pp. 865–879, Jun. 2022.
- [8] Z. Wang, Q. Dong, W. Guo, D. Li, J. Zhang, and W. Du, "Geometric imbalanced deep learning with feature scaling and boundary sample mining," *Pattern Recognit.*, vol. 126, 2022, Art. no. 108564.
- [9] H. Li, N. Wang, X. Ding, X. Yang, and X. Gao, "Adaptively learning facial expression representation via C-F labels and distillation," *IEEE Trans. Image Process.*, vol. 30, pp. 2016–2028, 2021.
- [10] W. Chen, K. Yang, Z. Yu, Y. Shi, and C. Chen, "A survey on imbalanced learning: Latest research, applications and future directions," *Artif. Intell. Rev.*, vol. 57, no. 6, pp. 1–51, 2024.
- [11] C. Elkan, "The foundations of cost-sensitive learning," in *Proc. Int. Joint Conf. Artif. Intell.*, 2001, pp. 973–978.
- [12] R. F. de Moraes and G. C. Vasconcelos, "Boosting the performance of over-sampling algorithms through under-sampling the minority class," *Neurocomputing*, vol. 343, pp. 3–18, 2019.
- [13] T. Zhu, X. Liu, and E. Zhu, "Oversampling with reliably expanding minority class regions for imbalanced data learning," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 6, pp. 6167–6181, Jun. 2023.
- [14] A. Islam, S. B. Belhaouari, A. U. Rehman, and H. Bensmail, "KNNOR: An oversampling technique for imbalanced datasets," *Appl. Soft Comput.*, vol. 115, 2022, Art. no. 108288.
- [15] H. Han, W.-Y. Wang, and B.-H. Mao, "Borderline-smote: A new over-sampling method in imbalanced data sets learning," in *Proc. Int. Conf. Intell. Comput.*, 2005, pp. 878–887.
- [16] G. Li, Z. Yu, K. Yang, M. Lin, and C. L. P. Chen, "Exploring feature selection with limited labels: A comprehensive survey of semi-supervised and unsupervised approaches," *IEEE Trans. Knowl. Data Eng.*, to be published, doi: 10.1109/TKDE.2024.3397878.
- [17] S. Barua, M. M. Islam, and K. Murase, "A novel synthetic minority oversampling technique for imbalanced data set learning," in *proc. 18th Int. Conf. Neural Inf. Process.*, Shanghai, China, 2011, pp. 735–744.
- [18] L. Ni, J. Li, H. Xu, X. Wang, and J. Zhang, "Fraud feature boosting mechanism and spiral oversampling balancing technique for credit card fraud detection," *IEEE Trans. Computat. Social Syst.*, vol. 11, no. 2, pp. 1615–1630, Apr. 2024.
- [19] Y. Zhang, G. Wang, X. Huang, and W. Ding, "Tsk fuzzy system fusion at sensitivity-ensemble-level for imbalanced data classification," *Inf. Fusion*, vol. 92, pp. 350–362, 2023.
- [20] M. Yan, S. C. Hui, and N. Li, "DML-PL: Deep metric learning based pseudo-labeling framework for class imbalanced semi-supervised learning," *Inf. Sci.*, vol. 626, pp. 641–657, 2023.
- [21] J. Xiong and J. Lai, "Similarity metric learning for RGB-infrared group re-identification," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 13662–13671.
- [22] C. Duan et al., "Multilevel similarity-aware deep metric learning for fine-grained image retrieval," *IEEE Trans. Ind. Informat.*, vol. 19, no. 8, pp. 9173–9182, Aug. 2023.
- [23] Z. Huang, X. Xu, J. Ni, H. Zhu, and C. Wang, "Multimodal representation learning for recommendation in Internet of Things," *IEEE Internet Things J.*, vol. 6, no. 6, pp. 10675–10685, Dec. 2019.
- [24] K. Q. Weinberger and L. K. Saul, "Distance metric learning for large margin nearest neighbor classification," *J. Mach. Learn. Res.*, vol. 10, no. 2, pp. 207–244, 2009.
- [25] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263–1284, Sep. 2009.
- [26] G. E. A. P. A. Batista, R. C. Prati, and M. C. Monard, "A study of the behavior of several methods for balancing machine learning training data," *SIGKDD Explor. Newsl.*, vol. 6, pp. 20–29, Jun. 2004.
- [27] X. Liu, J. Wu, and Z. Zhou, "Exploratory undersampling for class-imbalance learning," *IEEE Trans. Syst., Man, Cybern. B Cybern.*, vol. 39, no. 2, pp. 539–550, Apr. 2009.
- [28] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "Smote: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.
- [29] H. He, Y. Bai, E. A. Garcia, and S. Li, "Adasyn: Adaptive synthetic sampling approach for imbalanced learning," in *Proc. IEEE Int. Joint Conf. Neural Netw.*, 2008, pp. 1322–1328.
- [30] S. Barua, M. M. Islam, X. Yao, and K. Murase, "MWMOTE-majority weighted minority oversampling technique for imbalanced data set learning," *IEEE Trans. Knowl. Data Eng.*, vol. 26, no. 2, pp. 405–425, Feb. 2014.
- [31] L. Abdi and S. Hashemi, "To combat multi-class imbalanced problems by means of over-sampling techniques," *IEEE Trans. Knowl. Data Eng.*, vol. 28, no. 1, pp. 238–251, Jan. 2016.
- [32] L. Li, H. He, and J. Li, "Entropy-based sampling approaches for multi-class imbalanced problems," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 11, pp. 2159–2170, Nov. 2020.
- [33] A. Farshidvard, F. Hooshmand, and S. MirHassani, "A novel two-phase clustering-based under-sampling method for imbalanced classification problems," *Expert Syst. Appl.*, vol. 213, 2023, Art. no. 119003.
- [34] Y. Jin, Y. Dong, Y. Zhang, and X. Hu, "SSMD: Dimensionality reduction and classification of hyperspectral images based on spatial-spectral manifold distance metric learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5538916.
- [35] J. V. Davis, B. Kulis, P. Jain, S. Sra, and I. S. Dhillon, "Information-theoretic metric learning," in *Proc. 24th Int. Conf. Mach. Learn.*, 2007, pp. 209–216.
- [36] J. Goldberger, G. Hinton, S. Roweis, and R. Salakhutdinov, "Neighbourhood components analysis," in *Proc. Adv. Neural Inf. Process. Syst.*, 2005, vol. 17, pp. 513–520.
- [37] E. P. Xing, M. I. Jordan, S. J. Russell, and A. Y. Ng, "Distance metric learning with application to clustering with side-information," in *Proc. Adv. Neural Inf. Process. Syst.*, 2003, pp. 521–528.

- [38] W. Li, Y. Gao, L. Wang, L. Zhou, J. Huo, and Y. Shi, "OPML: A one-pass closed-form solution for online metric learning," *Pattern Recognit.*, vol. 75, pp. 302–314, 2018.
- [39] R. Jin, S. Wang, and Y. Zhou, "Regularized distance metric learning: Theory and algorithm," in *Proc. Adv. Neural Inf. Process. Syst.*, 2009, pp. 862–870.
- [40] P. Wu, S. C. H. Hoi, P. Zhao, C. Miao, and Z.-Y. Liu, "Online multi-modal distance metric learning with application to image retrieval," *IEEE Trans. Knowl. Data Eng.*, vol. 28, no. 2, pp. 454–467, Feb. 2016.
- [41] P. Jain, B. Kulis, I. S. Dhillon, and K. Grauman, "Online metric learning and fast similarity search," in *Proc. Adv. Neural Inf. Process. Syst.*, 2009, pp. 761–768.
- [42] H. Wu, Q. Zhou, R. Nie, and J. Cao, "Effective metric learning with co-occurrence embedding for collaborative recommendations," *Neural Netw.*, vol. 124, pp. 308–318, 2020.
- [43] Y. Li and D. Tao, "Online semi-supervised multi-task distance metric learning," in *Proc. IEEE 16th Int. Conf. Data Mining Workshops*, 2016, pp. 474–479.
- [44] W. Wu, D. Tao, H. Li, Z. Yang, and J. Cheng, "Deep features for person re-identification on metric learning," *Pattern Recognit.*, vol. 110, 2021, Art. no. 107424.
- [45] Y. Xu, Y. Yang, E. Wang, F. Zhuang, and H. Xiong, "Detect professional malicious user with metric learning in recommender systems," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 9, pp. 4133–4146, Sep. 2022.
- [46] D. Guan, J. Huang, A. Xiao, S. Lu, and Y. Cao, "Uncertainty-aware unsupervised domain adaptation in object detection," *IEEE Trans. Multimedia*, vol. 24, pp. 2502–2514, 2022.
- [47] G. Chechik, V. Sharma, U. Shalit, and S. Bengio, "Large scale online learning of image similarity through ranking," *J. Mach. Learn. Res.*, vol. 11, no. 3, pp. 1109–1135, 2010.
- [48] K. Song, F. Nie, J. Han, and X. Li, "Parameter free large margin nearest neighbor for distance metric learning," in *Proc. 31st AAAI Conf. Artif. Intell.*, 2017, pp. 2555–2561.
- [49] B. Nguyen, C. Morell, and B. De Baets, "Distance metric learning with the universum," *Pattern Recognit. Lett.*, vol. 100, pp. 37–43, 2017.
- [50] Y. Guo, Z. Cheng, J. Jing, Y. Lin, L. Nie, and M. Wang, "Enhancing factorization machines with generalized metric learning," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 8, pp. 3740–3753, Aug. 2022.
- [51] Y. Xu, C. Miao, Y. Liu, H. Song, Y. Hu, and H. Min, "Kernel-target alignment based non-linear metric learning," *Neurocomputing*, vol. 411, pp. 54–66, 2020.
- [52] S. Parameswaran and K. Q. Weinberger, "Large margin multi-task metric learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2010, pp. 1867–1875.
- [53] L. Gautheron, A. Habrard, E. Morvant, and M. Sebban, "Metric learning from imbalanced data," in *Proc. IEEE 31st Int. Conf. Tools Artif. Intell.*, 2019, pp. 923–930.
- [54] L. Feng, H. Wang, B. Jin, H. Li, M. Xue, and L. Wang, "Learning a distance metric by balancing kl-divergence for imbalanced datasets," *IEEE Trans. Syst., Man, Cybern. Syst.*, vol. 49, no. 12, pp. 2384–2395, Dec. 2019.
- [55] N. Wang, X. Zhao, Y. Jiang, and Y. Gao, "Iterative metric learning for imbalance data classification," in *Proc. 27th Int. Joint Conf. Artif. Intell.*, 2018, pp. 2805–2811.
- [56] B. Cao, Y. Liu, C. Hou, J. Fan, B. Zheng, and J. Yin, "Expediting the accuracy-improving process of SVMs for class imbalance learning," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 11, pp. 3550–3567, Nov. 2021.
- [57] W. Chen, K. Yang, Z. Yu, and W. Zhang, "Double-Kernel based class-specific broad learning system for multiclass imbalance learning," *Knowl.-Based Syst.*, vol. 253, 2022, Art. no. 109535.
- [58] W. Chen, K. Yang, W. Zhang, Y. Shi, and Z. Yu, "Double-Kernelized weighted broad learning system for imbalanced data," *Neural Comput. Appl.*, vol. 34, no. 22, pp. 19923–19936, 2022.
- [59] K. Yang, Z. Yu, C. L. P. Chen, W. Cao, J. You, and H.-S. Wong, "Incremental weighted ensemble broad learning system for imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 12, pp. 5809–5824, Dec. 2022.
- [60] K. Yang et al., "Hybrid classifier ensemble for imbalanced data," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 4, pp. 1387–1400, Apr. 2020.
- [61] J. Alcalá-Fdez et al., "Keel data-mining software tool: Data set repository, integration of algorithms and experimental analysis framework," *J. Mult.-Valued Log. Soft Comput.*, vol. 17, pp. 255–287, 2010.
- [62] A. Asuncion et al., "UCI machine learning repository," 2007.
- [63] T. Fawcett, "ROC graphs: Notes and practical considerations for researchers," *Mach. Learn.*, vol. 31, no. 1, pp. 1–38, 2004.
- [64] M. Friedman, "The use of ranks to avoid the assumption of normality implicit in the analysis of variance," *J. Amer. Statist. Assoc.*, vol. 32, no. 200, pp. 675–701, 1937.



Kaixiang Yang (Member, IEEE) received the BS degree from the University of Electronic Science and Technology of China in 2012, the MS degree from the Harbin Institute of Technology, China, in 2015, and the PhD degree from the School of Computer Science and Engineering, South China University of Technology, China, in 2020. He has been a research engineer with the 7th Research Institute, China Electronics Technology Group Corporation, Guangzhou, China, from 2015 to 2017, and has been a postdoctoral researcher with Zhejiang University from 2021 to 2023. He is currently an associate professor with the School of Computer Science and Engineering, South China University of Technology, Guangzhou. His research interests include pattern recognition, machine learning, and industrial data intelligence.



Zhiwen Yu (Senior Member, IEEE) received the PhD degree from the City University of Hong Kong in 2008. He is a professor with the School of Computer Science and Engineering, South China University of Technology, China. He serves as an associate editor of *IEEE Transactions on Systems, Man, and Cybernetics: Systems*. He has published more than 150 referred journal papers and international conference papers, including 50 IEEE Transactions papers.



Wuxing Chen received the MS degree from the Guangdong University of Technology, China, in 2023. He is currently working toward the PhD degree with the School of Future Technology, South China University of Technology, China. His research interests include machine learning and data mining.



Zefeng Liang received the MS degree with the School of Computer Science and Engineering, South China University of Technology. His research interests include machine learning and data mining.



C. L. Philip Chen (Fellow, IEEE) is the chair professor and dean with the College of Computer Science and Engineering, South China University of Technology. He is a fellow of AAAS, IAPR, CAA, and HKIE; a member of Academia Europaea (AE), European Academy of Sciences and Arts (EASA), and International Academy of Systems and Cybernetics Science (IASCYS). He obtained IEEE Norbert Wiener Award in 2018 for his contribution in systems and cybernetics, and machine learning. He is a highly cited Researcher by Clarivate Analytics from 2018–2020. His current research interests include cybernetics, systems, and computational intelligence. He received the 2016 Outstanding Electrical and Computer Engineers Award from his alma mater, Purdue University (in 1988), after he graduated from the University of Michigan at Ann Arbor, Ann Arbor, MI, USA in 1985. He was the IEEE Systems, Man, and Cybernetics Society President from 2012 to 2013, the editor-in-chief of the *IEEE Transactions on Systems, Man, and Cybernetics: Systems* (2014–2019), and currently, he is the editor-in-chief of the *IEEE Transactions on Cybernetics*, and an associate editor of the *IEEE Transactions on AI*, and *IEEE Transactions on Fuzzy Systems*. He was the Chair of TC 9.1 Economic and Business Systems of International Federation of Automatic Control from 2015 to 2017 and currently is a vice president with the Chinese Association of Automation (CAA).